অধ্যায় ২৯

Advanced JavaScript System Design, Microservices, Distributed Systems এবং Enterprise Architecture

২৯.১ Learning Outcomes

এই অধ্যায় শেষে আপনি functional ও non-functional requirements আলাদা করতে পারবেন, modular monolith ও microservices-এর trade-off ব্যাখ্যা করতে পারবেন, distributed transaction-এর Saga Pattern implement করতে পারবেন, idempotency, consistency, retries, observability, event delivery এবং failure recovery সম্পর্কে সিদ্ধান্ত নিতে পারবেন।

২৯.২ System Design আসলে কী?

System design শুধু diagram আঁকা নয়। একটি business requirement-কে নির্ভরযোগ্য, নিরাপদ, maintainable, observable ও cost-aware software system-এ রূপান্তর করার প্রক্রিয়া। ধরুন একটি travel platform-এ flight seat reservation, payment এবং booking confirmation একসঙ্গে করতে হবে। একটি service fail করলে অন্য service-এর state কী হবে—এটাই design-এর গুরুত্বপূর্ণ প্রশ্ন।

Functional requirements

- Customer package search করতে পারবে।
- Booking request submit করতে পারবে।
- Inventory reserve হবে।
- Payment success হলে booking confirmed হবে।
- Failure হলে reservation release হবে।

Non-functional requirements

- Availability, latency, security, durability, scalability, auditability, disaster recovery এবং operational cost।
- উদাহরণ: 95th percentile API latency 300ms-এর নিচে রাখতে চাইলে আগে traffic pattern ও dependency latency মাপতে হবে; 300ms কোনো universal guarantee নয়।

২৯.৩ Capacity Planning: অনুমান নয়, হিসাব

ধরা যাক প্রতিদিন ৮৬,৪০০ booking-related request আসে। Average throughput ≈ ১ request/second। কিন্তু peak traffic average-এর ২০ গুণ হলে ২০ request/second handle করতে হতে পারে।

```javascript
const dailyRequests = 86_400;
const avgRps = dailyRequests / (24 * 60 * 60);
const peakMultiplier = 20;
console.log({ avgRps, estimatedPeakRps: avgRps * peakMultiplier });
```

এটি শুধু starting estimate। Payload size, database queries, cache hit rate, concurrency, network, third-party payment latency ও retry amplification আলাদাভাবে বিবেচনা করতে হবে।

২৯.৪ Monolith, Modular Monolith ও Microservices

Monolith: একটি deployable application; সহজ deployment ও debugging, কিন্তু module boundaries দুর্বল হলে coupling বাড়ে। **Modular Monolith:** একই deployable-এর মধ্যে domain modules, interfaces এবং ownership পরিষ্কার। **Microservices:** service-গুলো independently deployable; scalability ও team autonomy বাড়তে পারে, কিন্তু network failure, distributed data, versioning এবং operational complexity বাড়ে।

Criterion	Modular Monolith	Microservices
Deployment	একটি unit	বহু unit
Local transactions	সহজতর	cross-service কঠিন
Observability	তুলনামূলক সহজ	tracing জরুরি
Team autonomy	মাঝারি	উপযুক্ত boundary হলে বেশি
Operational overhead	কম	বেশি

Recommendation: প্রমাণিত scale/team boundary না থাকলে modular monolith দিয়ে শুরু করা অনেক ক্ষেত্রে যৌক্তিক। Microservices কোনো automatic upgrade নয়।

২৯.৫ Domain-Driven Design এবং Bounded Context

Travel domain-এ `Catalog`, `Inventory`, `Booking`, `Payment`, `Notification` আলাদা bounded context হতে পারে। প্রত্যেকটির নিজস্ব language ও business rules থাকে। যেমন `BookingStatus` এবং `PaymentStatus` এক নয়। Database table অনুসারে নয়, business capability অনুসারে boundary নির্ধারণ করতে হবে।

```
Client / Web App
       |
   API Gateway
       |
 +-----+--------+---------+
 | Catalog      Booking   Identity |
 +-----+--------+---------+
                 |
           Saga Orchestrator
              /       \
        Inventory    Payment
              \       /
            Event Bus
                |
            Notification
```

এটি **conceptual diagram**, deployed topology নয়। Gateway authentication, authorization, rate limits এবং observability policy-এর বিকল্প নয়।

২৯.৬ Synchronous বনাম Asynchronous Communication

REST/HTTP synchronous request-response workflow-তে সহজ। Events asynchronous decoupling ও background work-এ সাহায্য করে। তবে event broker ব্যবহারে delivery guarantees, replay, schema evolution, ordering ও dead-letter handling যোগ হয়।

```javascript
const event = {
  eventId: 'evt-101',
  type: 'booking.confirmed.v1',
  aggregateId: 'booking-101',
  occurredAt: new Date().toISOString(),
  payload: { bookingId: 'booking-101' }
};
```

Event version পরিবর্তনের সময় পুরোনো consumers যেন অকারণে break না হয়, সে জন্য backward-compatible fields ও schema contract দরকার।

২৯.৭ Distributed Transactions এবং Saga Pattern

একটি local SQL transaction দিয়ে ভিন্ন service/database-এর সব change atomically commit করা যায় না। Saga-তে প্রতিটি local transaction-এর পর failure হলে compensating action চালানো হয়। Compensation একটি logical reversal; সব external side effect নিখুঁতভাবে undo করা যায় না।

Happy path

1. Booking request validate।
2. Inventory seat reserve।
3. Payment charge।

4. Booking confirm।
5. Notification event publish।

Failure path

Payment fail করলে Inventory release করতে হবে। Inventory release-ও fail করলে `MANUAL_REVIEW` বা reconciliation queue প্রয়োজন।

```
Booking Requested
       |
  Reserve Inventory ----failure----> Reject
       |
  Charge Payment ------failure-----> Release Inventory
       |
  Confirm Booking
       |
  Publish Event
```

Orchestration-এ একটি coordinator workflow পরিচালনা করে। **Choreography**-তে services events শুনে নিজেদের action নেয়। Orchestration traceability সহজ করতে পারে; choreography coupling কমালেও emergent workflow বোঝা কঠিন হতে পারে।

২৯.৮ Idempotency এবং Duplicate Request

Network timeout মানেই operation fail নয়। Client একই request retry করলে duplicate charge বা duplicate booking হতে পারে। Idempotency key ব্যবহার করে একই logical request-এর ফলাফল reuse করতে হয়। একই key দিয়ে ভিন্ন payload এলে conflict ফিরিয়ে দিতে হবে।

```javascript
const idempotencyKey = 'trip-101';
// Server must store key + request fingerprint + result atomically.
```

In-memory `Map` একটি learning tool; production-এ durable unique constraint ও transaction দরকার। Concurrent requests-এ per-process lock যথেষ্ট নয় যখন একাধিক replica থাকে।

২৯.৯ Consistency, CAP এবং Data Ownership

Distributed systems-এ latency, partition, availability ও consistency-এর trade-off আছে। CAP-কে "যেকোনো দুটি বেছে নাও" বলে সরলীকরণ করা ভুল; এটি network partition চলাকালে নির্দিষ্ট consistency ও availability trade-off বোঝায়।

Inventory-এর মতো scarce resource-এ strong coordination প্রয়োজন হতে পারে। Analytics ও notification eventual consistency সহ্য করতে পারে। প্রত্যেক service-এর database ownership আলাদা হলে অন্য service সরাসরি table modify করবে না।

২৯.১০ Reliability Patterns

- **Timeout:** অনির্দিষ্ট অপেক্ষা এড়ায়।
- **Retry with exponential backoff and jitter:** transient error সামলায়; permanent validation failure retry নয়।
- **Circuit breaker:** failing dependency-তে অপ্রয়োজনীয় load কমায়।
- **Bulkhead:** একটি failure যেন সব resources consume না করে।
- **Dead-letter queue:** repeatedly failing messages investigation-এর জন্য রাখে।
- **Transactional outbox:** business state change ও event record একই database transaction-এ commit করে; পরে publisher broker-এ পাঠায়।
- **Inbox/deduplication:** at-least-once delivery-তে duplicate processing প্রতিরোধ করে।

```javascript
function backoffMs(attempt, base = 100, cap = 5_000) {
  const exponential = Math.min(cap, base * 2 ** attempt);
  return Math.floor(Math.random() * exponential);
}
```

Retry-এর আগে operation idempotent কি না এবং request deadline শেষ হয়েছে কি না যাচাই জরুরি।

২৯.১১ Observability ও Security

Metrics দিয়ে rate/errors/latency বোঝা যায়; structured logs দিয়ে ঘটনা অনুসন্ধান করা যায়; distributed traces দিয়ে cross-service request path দেখা যায়। Trace ID লগে থাকলেও password, token, payment card বা private customer data লগ করা যাবে না। Service-to-service authorization, least privilege, TLS, secret rotation এবং audit trail প্রয়োজন।

২৯.১২ Practical Project — TravelSaga Booking Orchestrator

Goal: একটি in-memory educational Saga তৈরি করা, যেখানে inventory reservation, payment charge, booking confirmation, rollback ও idempotency দেখানো হবে।

Project Structure

```
source-code/chapter-29/
├── package.json
├── saga.mjs
├── demo.mjs
├── README.md
└── test/
    └── saga.test.mjs
```

Run

```
TERMINAL
cd source-code/chapter-29
node --test
node demo.mjs
```

Core API

```
JAVASCRIPT
import {
  createInventory,
  createPaymentGateway,
  createBookingRepository,
  createTravelSaga
} from './saga.mjs';

const inventory = createInventory({ seats: 2 });
const payments = createPaymentGateway();
const bookings = createBookingRepository();
const saga = createTravelSaga({ inventory, payments, bookings });

const booking = await saga.book({
  id: 'trip-101', customer: 'Rahim', amount: 12500000
});
console.log(booking.status); // CONFIRMED
```

Code walkthrough: `createInventory()` seat reservation ও release পরিচালনা করে; `createPaymentGateway()` charge/refund simulate করে; `createBookingRepository()` confirmed booking রাখে; `createTravelSaga()` compensation stack reverse order-এ চালায়। একই id-র concurrent calls single-flight করে এবং repeat call-এ saved result দেয়।

Money: `amount` integer minor units (যেমন পয়সা) হিসেবে গণ্য; floating-point currency arithmetic এড়ানোর জন্য।

Limitations: এটি real microservices নয়; in-memory state restart-এ হারায়। Crash between charge and confirmation, external payment uncertainty, cross-instance race, durable outbox, payment reconciliation ও distributed lock এই demo-তে implement করা হয়নি। Production use-এর আগে এগুলো অপরিহার্য।

২৯.১৩ Failure Scenario Lab

১. `shouldFail: () => true` দিয়ে payment failure simulate করুন; inventory পুনরুদ্ধার হয়েছে কি না দেখুন।
২. `seats: 0` দিয়ে sold-out case পরীক্ষা করুন; কোনো charge হয়নি কি না দেখুন।
৩. একই booking ID দুইবার পাঠিয়ে seat count অপরিবর্তিত আছে কি না যাচাই করুন।
৪. একই ID কিন্তু ভিন্ন amount পাঠিয়ে `IDEMPOTENCY_CONFLICT` দেখুন।
৫. `Promise.all()` দিয়ে concurrent duplicate requests পাঠান।
৬. Compensation function-এ artificial error inject করে manual reconciliation-এর প্রয়োজন বোঝার চেষ্টা করুন।

২৯.১৪ Common Mistakes

১. Network timeout-কে নিশ্চিত failure ধরে আবার charge করা।
২. Microservices নিলেই scalability নিশ্চিত হবে মনে করা।
৩. Database ownership ভেঙে অন্য service-এর table সরাসরি update করা।
৪. At-least-once events-এ consumer idempotency না রাখা।
৫. Business operation-এর পরে event publish ব্যর্থ হলে outbox ছাড়া state mismatch তৈরি করা।
৬. Retry-তে jitter বা upper bound না রাখা।
৭. Compensation fail হলে error গোপন করা।
৮. Distributed trace context propagate না করা।
৯. PII ও credentials logs-এ লেখা।
১০. Production SLA নির্ধারণের আগে measurement না করা।

২৯.১৫ Exercises

১. Modular monolith এবং microservices-এর তিনটি trade-off লিখুন।
২. Booking, Payment ও Inventory-এর bounded context আলাদা করুন।
৩. Saga-র happy path এবং payment-failure path আঁকুন।
৪. Idempotency key ও request fingerprint-এর কাজ ব্যাখ্যা করুন।
৫. `at-least-once delivery`-তে duplicate handling কেন প্রয়োজন?
৬. Transactional outbox pattern-এর কাজ লিখুন।
৭. ৪৩২,০০০ daily requests হলে average RPS কত?
৮. Retry storm কেন বিপজ্জনক?
৯. Inventory strong coordination চাইতে পারে, কিন্তু notification eventual consistency সহ্য করে—কেন?
১০. TravelSaga-তে durable database ও reconciliation worker যোগ করার architecture proposal লিখুন।

Sample Solutions

১. Deployment autonomy বনাম operational complexity; independent scaling বনাম network overhead; local transaction বনাম distributed consistency।
২. Booking booking lifecycle, Payment charge/refund, Inventory available/reserved seats-এর মালিক।
৩. Success: reserve→charge→confirm; failure: reserve→charge fail→release।
৪. Key logical request চিহ্নিত করে; fingerprint একই key-তে payload পরিবর্তন শনাক্ত করে।
৫. Broker একই event পুনরায় deliver করতে পারে; duplicate side effects এড়াতে deduplication দরকার।
৬. Business change ও outbox row একই transaction-এ commit; publisher পরে event পাঠায়।
৭. `432000 / 86400 = 5 RPS`।
৮. Failure-এর সময় repeated requests downstream overload বাড়ায়।
৯. Overselling ঠেকাতে inventory coordination; delayed notification business invariant ভাঙে না।
১০. Unique idempotency constraint, transactional booking/outbox tables, async worker, retry policy, audit logs, manual review এবং reconciliation schedule।

২৯.১৬ MCQ — Final Check

১. Saga কী? A) distributed workflow with compensation B) CSS library C) encryption D) browser storage
২. Idempotency কী প্রতিরোধ করে? A) duplicate side effects B) all latency C) every network failure D) schema migration
৩. Outbox কেন? A) state/event consistency B) CSS minification C) token creation D) image compression
৪. Circuit breaker কী করে? A) failing dependency calls সীমিত করে B) database encrypt করে C) UI render করে D) package install করে
৫. Bounded context কী নির্দেশ করে? A) domain boundary B) TCP port C) Git branch D) memory size
৬. Eventual consistency কী? A) replicas সময় নিয়ে converge করে B) always immediate C) no data D) no failures
৭. Compensation কী? A) semantic undo action B) process kill C) memory leak D) CDN
৮. Trace ID কী কাজে লাগে? A) cross-service correlation B) password hashing C) SQL index D) CSS theme
৯. Dead-letter queue কী রাখে? A) repeatedly failed messages B) browser cache C) JWT secrets D) image thumbnails
১০. Distributed system-এ retry করার আগে কী জরুরি? A) idempotency ও deadline B) font selection C) color scheme D) no logs

Answers ও ব্যাখ্যা: ১-A (local steps + compensation), ২-A (same request repeated হলেও double charge এড়ানো), ৩-A (database change ও event publication gap কমানো), ৪-A (cascade failure সীমিত), ৫-A (business model ownership), ৬-A (replicas eventually converge), ৭-A (previous step-এর logical reversal), ৮-A (distributed request debugging), ৯-A (investigation/reprocessing), ১০-A (unsafe retries নতুন সমস্যা তৈরি করতে পারে)।

২৯.১৭ Architecture Review Checklist

- ☐ Requirements, traffic estimates ও failure assumptions documented
- ☐ Domain boundaries এবং data ownership defined
- ☐ Authentication/authorization ও secrets strategy
- ☐ Idempotency, retries, timeouts এবং compensations
- ☐ Event schema/versioning ও outbox/inbox plan
- ☐ Monitoring, alerting, tracing ও incident runbooks
- ☐ Backup, disaster recovery, reconciliation
- ☐ Load, integration, chaos এবং security testing plan

২৯.১৮ Recap ও Next Chapter

এই অধ্যায়ে scalable system design-এর বাস্তব trade-offs, microservice boundaries, distributed consistency, Saga, idempotency এবং resilience শিখলাম। পরবর্তী **Chapter 30**-এ full-stack capstone ও career roadmap-এর মাধ্যমে বইয়ের সব দক্ষতা একত্র করা হবে।

Further reading: Martin Fowler-এর microservices ও patterns writings; Microsoft Azure Architecture Center-এর Saga/Outbox patterns; AWS Builders Library-এর retries/timeouts/idempotency; OpenTelemetry documentation। Version-specific guidance অফিসিয়াল উৎস দিয়ে যাচাই করতে হবে।